SW개발/HW제작 설계서

프로젝트 명 : VLM 기반 Physical AI 지뢰탐지 로봇 시스템

2026. 09. 07

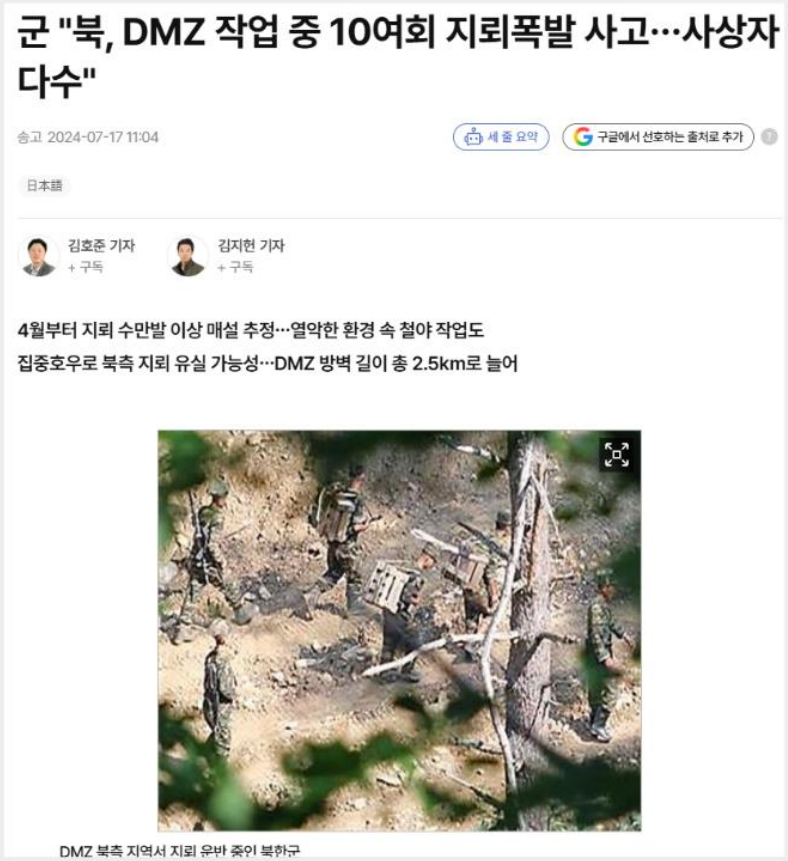
티놀로지

수행 단계별 주요 산출물

단계	산출물	일반 ·모바일 APP ·Web 등	응용 소프트웨어 ·빅데이터 ·인공지능 ·블록체인 등	응용 하드웨어 ·IoT ·로봇 ·드론 등
환경 분석	시장/기술 환경 분석서	△	△	△
	설문조사 결과서	△	△	△
	인터뷰 결과서	△	△	△
요구사항 분석	요구사항 정의서	○	○	○
	유즈케이스 정의서	△	△	△
아키텍처 설계	서비스 구성도(시스템 구성도)	○	○	○
	서비스 흐름도(데이터 흐름도)	△	○	△
	UI/UX 정의서	△	△	△
	하드웨어/센서 구성도	-	-	○
기능 설계	메뉴 구성도	○	○	○
	화면 설계서	○	○	△
	엔티티 관계도	○	○	△
	기능 처리도(기능 흐름도)	○	○	○
	알고리즘 명세서/설명서	△	○	○
	데이터 수집처리 정의서	-	○	-
	하드웨어 설계도	-	-	○
개발 / 구현	프로그램 목록	○	○	○
	테이블 정의서	○	○	△
	핵심 소스코드	○	○	○

※ ○ 필수, △ 선택

국내 지뢰 위험 동향



군 "북, DMZ 작업 중 10여회 지뢰폭발 사고…사상자 다수"

송고 2024-07-17 11:04

세 줄 요약 | 구글에서 선호하는 출처로 추가

日本語

김호준 기자 + 구독 | 김지헌 기자 + 구독

4월부터 지뢰 수만발 이상 매설 추정…열악한 환경 속 철야 작업도

집중호우로 북측 지뢰 유실 가능성…DMZ 방벽 길이 총 2.5km로 늘어

DMZ 북측 지역서 지뢰 유발 중인 북한군

- 지속되는 DMZ 지뢰 위협

- 2024년 4월부터 DMZ 북측 지역에서 지뢰 매설 작업이 지속되고 있으며 매설된 지뢰는 수만 발 이상으로 추정하고 있다.
- 집중호우가 발생하면 공유하천을 통해 북측 지뢰가 남측 지역으로 유입될 가능성도 있어 위험 범위가 확대될 수 있다.

- 현장 작업자의 높은 사고 위험

- 지뢰 매설 작업 중 10여 차례의 폭발 사고가 발생하여 다수의 사상자가 발생하는 등 작업자의 안전이 크게 위협받고 있다.
- 작업자가 지뢰 탐지와 제거를 위해 위험지역에 직접 진입하는 기존 방식은 폭발 사고와 추가 인명 피해로 이어질 가능성이 크다.

| 시장/기술 동향 분석

국외 지뢰 피해 동향

미얀마에서만 2,000명 피해... 세계 지뢰 사상자 4년 만 최다

허경주 · 2025. 12. 2. 16:32

0

내전·전쟁 겪은 시리아·우크라이나도 사고
"민간인을 지뢰 제거용 인간 방패로 사용"



- 확대되는 세계 지뢰 피해

- 2024년 전 세계 52개국에서 지뢰와 불발탄으로 인해 총 6,279명의 사상자가 발생하였다.
- 미얀마에서만 2,029명의 피해자가 발생했으며 시리아, 아프가니스탄, 우크라이나 등에서도 지뢰 사고가 지속되고 있다.

- 민간인 중심의 지속적인 피해

- 전체 피해자의 약 90%가 민간인으로 여성과 어린이가 피해자의 절반 가량을 차지한다.
- 전쟁이 종료된 지역에서도 매설된 지뢰와 불발탄이 장기간 남아 주민의 이동과 일상생활을 위협하고 있다.

국내 지뢰 제거 로봇 기술 동향



'폭발물 탐지제거 로봇' 軍 실전배치…국내 개발 로봇으론 처음

송고 2026-03-05 11:00

세 줄 요약 | 구글에서 선호하는 출처로 추가

中文

김철선 기자
+ 구독

DMZ 지뢰지대 통로 개척 임무 등 장병 대신 수행

- 국내 최초 폭발물 제거 로봇 전력화

- 국내 기술로 개발된 KR1이 군에 실전 배치되면서 지뢰 및 폭발물 제거 작업의 로봇화가 본격적으로 진행되고 있다.
- 기존에는 작업자가 보호장비를 착용하고 현장에 직접 진입했지만 로봇이 위험 임무를 대신 수행하여 인명 피해를 줄일 수 있다.

- 원격조작 기반의 다목적 임무 수행

- 원격조작을 통해 DMZ 지뢰지대 통로 개척, 폭발물 탐지/제거 및 위험 지역 정찰 등의 임무를 수행한다.
- 국내 지뢰 제거 기술이 작업자 중심 방식에서 로봇 기반 원격 대응 방식으로 전환되고 있음을 보여준다.

| 시장/기술 동향 분석

기존 제품 비교 및 본 프로젝트의 차별성

	주요 목적	상황 인식	안전 안내	조작 방식	시스템 연계
기존 폭발물 탐지/제거 로봇	위험지역 정찰 및 지뢰/폭발물의 원격 탐지/제거	작업자가 카메라와 센서 정보를 확인하여 상황을 판단	원격작업을 통해 제거 작업자의 현장 투입을 최소화	전용 조종장치를 이용한 원격제어 중심	주행·정찰·폭발물 처리 등 군사 임무 수행에 중점
본 프로젝트	지뢰 의심 물체 탐지·제거와 주변 인원 보호	VLM이 영상 속 사람과 주변 위험 상황을 분석	사람 감지 시 인상착의를 안내하고 위험지역 이탈 음성을 출력	마스터 로봇팔의 움직임을 슬레이브 로봇팔에 실시간 전달	원격주행·VLM 분석/음성 경고/텔레오퍼레이션을 하나의 흐름으로 통합

단순한 원격 제거를 넘어 위험 상황을 이해하고 사람에게 경고하며 실제 대응까지 수행하는 지능형 원격 로봇 시스템

| 요구사항 정의서

요구사항ID	요구사항명	기능ID	기능명	세부사항	예외사항
A01	모바일 플랫폼 원격주행	A01_B01	조작 명령 입력	상황실의 주행 명령 입력으로 모바일 플랫폼의 주행을 제어	입력이 없으면 정지 상태를 유지
		A01_B02	주행 제어	전진/후진/좌회전/우회전 및 정지 동작을 수행	비정상적인 제어 명령 수신 시 해당 명령을 무시
		A01_B03	주행 속도 조절	작업 환경과 장애물 거리에 따라 모바일 플랫폼의 주행 속도를 조절	설정된 최대속도를 초과하는 명령을 제한
		A01_B04	통신 두절 정지	상황실과의 통신이 일정 시간 이상 끊기면 플랫폼을 자동으로 정지	통신 복구 후 작업자의 재시작 명령을 받아 운행
A02	현장 영상 및 상황 인식	A02_B01	실시간 영상 전송	Depth Camera로 촬영한 현장 영상을 상황실 화면에 실시간으로 전송	영상 수신 실패 시 카메라 연결 상태를 작업자에게 알림
		A02_B02	거리정보 측정	Depth Camera를 활용하여 사람과 주변 장애물까지의 거리를 측정	거리 측정값이 유효하지 않으면 영상 정보만 제공
		A02_B03	사람 인식	현장 영상에서 위험지역에 접근한 사람을 실시간으로 탐지	사람이 탐지되지 않으면 대기 상태를 유지
		A02_B04	VLM 상황 분석	VLM을 활용하여 감지된 사람의 인상착의와 주변 위험 상황을 분석	-
		A02_B05	분석 결과 표시	사람 탐지 여부와 VLM 분석 결과를 상황실 화면에 제공	VLM 연결 실패 시 실시간 영상만 제공
A03	위험지역 음성 경고	A03_B01	경고 문장 생성	VLM 분석 결과를 바탕으로 사람의 인상착의와 위험지역을 벗어나도록 안내	VLM 분석 실패 시 사전에 설정한 기본 경고 문장을 사용
		A03_B02	TTS 음성 출력	생성된 안내 문장을 음성으로 변환하여 현장 스피커로 출력	-
		A03_B03	반복 경고	사람이 위험지역에 계속 머무르면 일정한 간격으로 음성 경고를 반복	사람이 위험지역에서 벗어나면 경고 출력을 중지

| 요구사항 정의서

요구사항ID	요구사항명	기능ID	기능명	세부사항	예외사항
A04	로봇팔 텔레오퍼레이션	A04_B01	마스터 조작값 취득	상황실에 설치된 마스터 로봇팔에서 각 관절의 위치값을 실시간으로 취득	관절값이 정상 범위를 벗어나면 해당 입력을 제한
		A04_B02	조작값 원격 전송	마스터 로봇팔의 관절값을 슬레이브 로봇팔로 전송	데이터 전송이 지연되거나 중단되면 슬레이브 로봇팔을 정지
		A04_B03	슬레이브 동작 동기화	수신한 관절값을 변환하여 슬레이브 로봇팔이 마스터 로봇팔의 움직임을 추종	초기 위치가 일치하지 않으면 동기화를 시작하지 않음
		A04_B04	듀얼 로봇팔 개별 제어	각각의 로봇팔을 독립적으로 원격 조작	-
		A04_B04	모의 지뢰 제거	로봇팔의 그리퍼로 위험 매설물을 파지하여 지정된 위치로 이동	-
A05	원격 통신 및 모니터링	A05_B01	제어 명령 송수신	IP를 통해 주행 명령, 로봇팔 관절값과 장치 제어 명령을 송수신	통신 지연이 기준값을 초과하면 장치 동작을 일시 정지
		A05_B02	현장정보 수신	카메라 영상, 지뢰 탐지 신호와 시스템 동작 상태를 상황실로 전송	-
		A05_B03	장치 상태 모니터링	모바일 플랫폼, 카메라, 센서와 로봇팔의 연결 및 동작 상태를 실시간으로 확인	-
A06	시스템 안전관리	A06_B01	시스템 초기화	주행부, 카메라, 센서와 로봇팔의 연결 상태 및 초기 위치를 확인	초기화가 완료되지 않으면 전체 작업을 시작하지 않음
		A06_B02	동작 범위 제한	로봇팔의 관절각과 모바일 플랫폼의 최대속도를 설정 범위 이내로 제한	허용 범위를 벗어난 명령은 수행하지 않음

| 요구사항 정의서

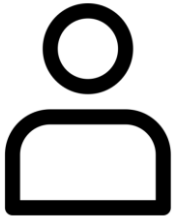
구분	기능	설명
S/W	모바일 플랫폼 원격주행	상황실에서 모바일 플랫폼의 전진/후진/회전/정지를 제어할 수 있다.
	실시간 영상 모니터링	Depth Camera로 촬영한 현장 영상을 상황실에서 실시간으로 확인할 수 있다.
	거리 및 장애물 인식	Depth 정보를 활용하여 사람과 주변 장애물까지의 거리를 측정할 수 있다.
	VLM 기반 사람 인식	현장 영상에서 위험지역에 접근한 사람을 인식할 수 있다.
	인상착의 및 상황 분석	VLM을 통해 감지된 사람의 인상착의와 주변 위험 상황을 분석할 수 있다.
	위험지역 음성 경고	민간인 감지 시 위험지역에서 벗어나도록 안내 문장을 생성하고 음성으로 출력할 수 있다.
	마스터/슬레이브 제어	마스터 로봇팔의 관절값을 슬레이브 로봇팔에 전달하여 움직임을 동기화할 수 있다.
	듀얼 로봇팔 제어	왼쪽 로봇팔과 오른쪽 로봇팔을 각각 독립적으로 원격 조작할 수 있다.
	원격 통신 및 상태 확인	주행/로봇팔 제어 명령과 영상/센서 정보를 송수신하고 실시간 장치 상태를 확인할 수 있다.
	비상 정지 및 안전제어	통신 두절이나 이상 상황 발생 시 모바일 플랫폼과 로봇팔의 동작을 정지할 수 있다.

구분	기능	설명
H/W	모바일 플랫폼	작업자를 대신하여 위험지역으로 이동하고 탐지/제거 장치를 운반할 수 있다.
	주행 모터 및 드라이버	모바일 플랫폼의 전진/후진/회전/속도를 제어할 수 있다.
	상황실 마스터 조작부	두 대의 마스터 로봇팔을 이용하여 현장 시스템을 원격 조작할 수 있다.
	슬레이브 로봇팔	마스터 로봇팔의 움직임을 따라 모의 지뢰 제거 작업을 수행할 수 있다.
	제거용 그리퍼	모의 지뢰 또는 위험 매설물을 파지하여 지정된 위치로 이동할 수 있다.
	Depth Camera	현장 영상과 거리정보를 취득하여 상황실과 VLM에 제공할 수 있다.
	음성 출력 스피커	VLM 분석 결과와 위험지역 이탈 안내를 현장에 음성으로 출력할 수 있다.
	로봇팔 통신 장치	U2D2를 통해 로봇팔의 관절값과 제어 명령을 송수신할 수 있다.
	현장 제어 및 통신부	제어기와 네트워크 장치를 이용하여 센서/주행부/로봇팔을 통합 제어할 수 있다.

| 유즈케이스 정의서



상황실



주행/영상 확인/로봇팔 조작

시스템 연결 및 초기화
카메라/플랫폼/로봇팔 상태 확인

모바일 플랫폼 원격주행
작업자가 상황실에서 위험지역으로 이동

실시간 현장 영상 및 거리확인
작업자가 주변 상황과 이동 경로를 판단

VLM 사람/위험상황 분석
현장 영상에서 접근자와 주변 상황 분석

사람 감지 시
인상착의 안내 및 음성 경고

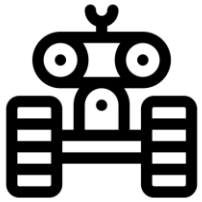
사람 미감지 시
작업을 계속 진행

제거 대상 육안 확인
모의 지뢰 또는 위험 매설물 위치 확인

로봇팔 텔레오퍼레이션
마스터 움직임을 슬레이브 로봇팔에 전달

모의 지뢰 원격 제거
그리퍼로 파지하여 배선 제거

작업 완료 확인 및 복귀
상태 확인 후 안전 위치로 이동



모바일
플랫폼

VLM 분석 시스템
사람/안상착의/위험상황 분석



위험지역 접근자
현장 음성 경고 수신

| 유즈케이스 정의서

항목	설명		
개요	작업자가 상황실에서 모바일 플랫폼을 원격으로 주행시키고 현장 영상과 센서 정보를 확인하여 지뢰 의심 지점을 탐지한 뒤 텔레오퍼레이션으로 모의 지뢰를 제거한다.		
관련엑터	주 엑터	상황실 작업자	
	보조 엑터	위험지역 접근자, 모바일 플랫폼, Depth Camera, VLM 분석 시스템, 지뢰 탐지 센서, 마스터/슬레이브 로봇팔, 음성 출력 장치	
우선 순위	상	중요도	3
		난이도	3
선행 조건	상황실과 현장 시스템의 전원 및 네트워크가 연결되고 카메라/탐지 센서/모바일 플랫폼/로봇팔의 초기화가 완료되어야 한다.		
후행 조건	탐지 및 제거 결과가 상황실 화면에에 나타나고 위험 매설물 지역으로 유추되는 곳으로 이동한 후 모바일 플랫폼과 로봇팔이 안전 상태로 정지해야 한다.		
시나리오	기본 시나리오	1. 작업자는 상황실에서 전체 장치의 연결 및 동작 상태를 확인한다. 2. 모바일 플랫폼을 위험지역으로 원격 이동시킨다. 3. Depth Camera 영상과 거리정보를 통해 주변 환경과 장애물을 확인한다. 4. VLM은 현장 영상에서 사람과 주변 위험 상황을 분석한다. 5. 사람이 감지되면 인상착의를 상황실에 안내하고 현장 스피커로 민간인이 위험지역에서 벗어나도록 경고한다. 6. 작업자는 상황실에서 마스터 로봇팔을 조작하여 매설물을 수색한다. 7. 작업자는 마스터 로봇팔을 조작하여 슬레이브 로봇팔을 의심 물체에 접근시킨다. 8. 그리퍼로 모의 지뢰를 파지하고 지뢰의 전원이 종료된 것을 확인한다. 9. 작업 결과를 확인한 후 로봇팔과 모바일 플랫폼을 안전 위치로 복귀시킨다.	
	대안 시나리오	**A1: 위험지역에서 사람 감지 시** 1. VLM이 사람의 위치와 인상착의를 분석한다. 2. 사람이 위험지역에서 벗어날 때까지 일정한 간격으로 음성 경고를 출력한다. **A2: 통신 지연 또는 연결 끊김 시** 1. 모바일 플랫폼과 슬레이브 로봇팔의 동작을 즉시 정지한다. 2. 통신 재연결 후 작업자가 장치 상태를 확인하고 작업을 재개한다. **A3: VLM 분석 실패 시** 1. 상황실에 분석 오류를 표시하고 실시간 영상만 제공한다. **A4: 모의 지뢰 파지 실패 시** 1. 파지 상태를 확인하며 제거 작업을 재시도한다.	

| 유즈케이스 정의서

비기능적 요구사항	**실시간성**	현장 영상, 센서 정보와 원격조작 명령이 작업자가 연속적으로 제어할 수 있는 수준으로 전송되어야 한다.
	안전성	통신 두절, 장치 이상 또는 비상 정지 명령 발생 시 모바일 플랫폼과 로봇팔이 즉시 정지해야 한다.
	신뢰성	카메라/센서/로봇팔의 연결 상태를 지속적으로 확인하고 이상 발생 시 작업자에게 알려야 한다.
	사용성	작업자가 상황실에서 영상, VLM 분석 결과와 장치 상태를 쉽게 확인할 수 있어야 한다.
	확장성	카메라 및 로봇팔을 변경하거나 추가할 수 있도록 장치별 기능을 모듈화해야 한다.

| 서비스 구성도 - 서비스 시나리오

모바일 플랫폼 원격 주행

1) 작업자는 상황실에서 모바일 플랫폼과 카메라의 연결 상태를 확인한다.
2) 상황실에서 전진/후진/좌회전/우회전 및 정지 명령을 입력한다.
3) 입력된 주행 명령은 동일한 IP를 통해 현장 제어기로 전송된다.
4) 현장 제어기는 모터 드라이버를 제어하여 모바일 플랫폼을 이동시킨다.
5) 통신이 끊기거나 문제 발생 시 즉시 정지한다.

실시간 현장 모니터링

1) Depth Camera가 모바일 플랫폼 주변의 영상과 거리 정보를 취득한다.
2) 취득한 현장 정보를 동일한 IP를 통해 상황실로 전달한다.
3) 작업자는 모니터에서 이동 경로와 주변 장애물을 실시간으로 확인한다.
4) 현장 영상을 바탕으로 플랫폼의 이동 방향과 접근 위치를 결정한다.

VLM 기반 현장 상황 분석

1) Depth Camera가 위험 지역의 현장을 실시간으로 촬영한다.
2) 촬영된 영상은 VLM 분석 시스템으로 전달된다.
3) VLM은 영상에서 사람의 접근 여부와 인상착의 및 주변 상황을 분석한다.
4) 분석 결과는 상황실 모니터에 표시되어 작업자의 상황 판단을 지원한다.
5) 최종 주행 및 로봇팔 조작 여부는 작업자가 현장 영상을 확인한 후 결정한다.

AI 음성 위험 경고

1) VLM이 위험지역에 접근한 사람을 감지한다.
2) 감지된 사람의 인상착의와 위험지역 이탈 안내 문장을 생성한다.
3) 생성된 문장을 TTS를 이용하여 AI 음성으로 변환한다.
4) 모바일 플랫폼의 스피커를 통해 해당 지역에서 벗어나도록 안내한다.
5) 사람이 위험지역에 계속 머무르면 일정한 간격으로 경고를 반복한다.

| 서비스 구성도 - 서비스 시나리오

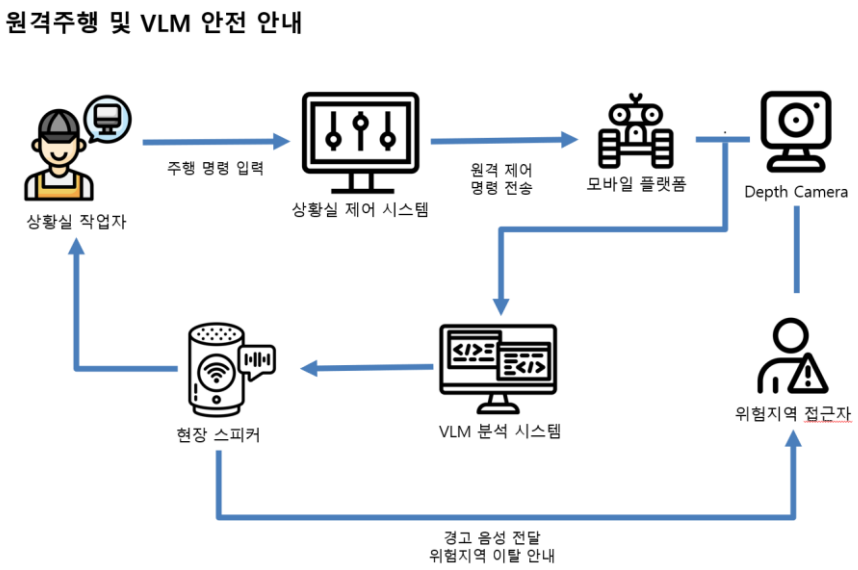
로봇팔 텔레오퍼레이션

1) 작업자는 상황실에서 실시간 현장 영상을 확인한다.
2) 두 대의 마스터 로봇팔을 움직여 원격 작업 명령을 입력한다.
3) 각 관절의 위치값을 U2D2를 통해 상황실 제어 PC에서 받는다.
4) 취득한 관절값을 네트워크를 통해 현장 제어 PC로 전송한다.
5) 현장 슬레이브 로봇팔은 수신한 관절값에 따라 마스터 로봇팔의 움직임을 실시간으로 추종한다.
6) 작업자는 두 로봇팔을 개별적으로 조작하여 대상 접근/고정 및 배선 처리 작업을 수행한다.

모의 위험물 원격 처리

1) 작업자는 상황실에서 모의 위험물의 배선 도면과 현장 영상을 확인한다.
2) 슬레이브 로봇팔을 원격 조작하여 제거 대상 배선에 접근한다.
3) 그리퍼를 이용하여 도면에 지정된 위험 배선을 분리한다.
4) 배선이 정상적으로 분리되면 LCD와 LED가 꺼진다.
5) 작업자는 작동 해제 상태를 확인하고 필요하면 모의 위험물을 지정된 위치로 이동한다.
6) 작업 완료 후 로봇팔과 모바일 플랫폼을 안전 위치로 복귀시킨다.

| 서비스 흐름도



원격주행 및 현장 모니터링

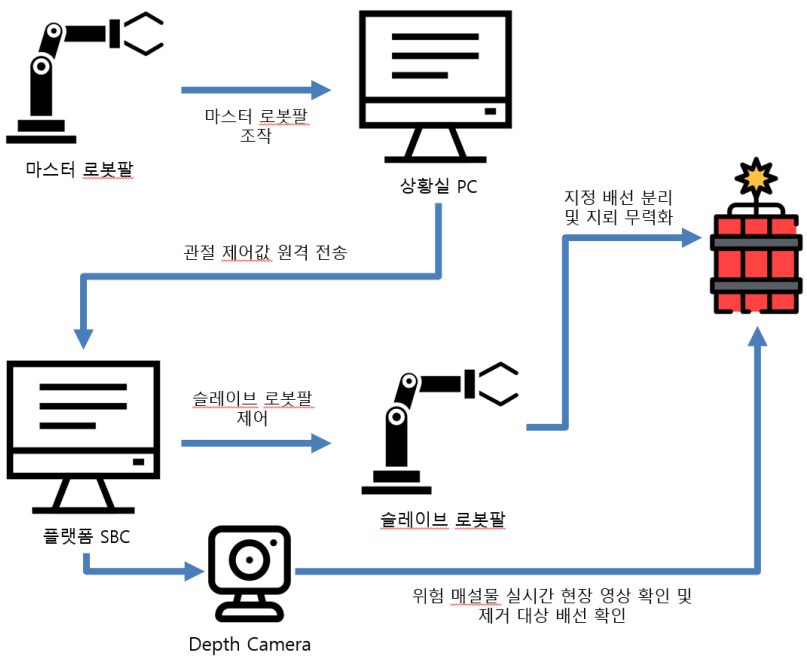
1) 작업자는 상황실에서 모바일 플랫폼의 주행 명령을 입력한다.
2) 상황실 제어 시스템은 입력된 주행 명령을 현장 모바일 플랫폼에 전달한다.
3) 모바일 플랫폼은 명령에 따라 위험지역으로 이동하고 현재 동작상태를 상황실로 전송한다.
4) Depth Camera로 촬영한 현장 영상과 거리정보를 상황실 모니터에 실시간으로 제공한다.

VLM 기반 상황 분석 및 음성 경고

1) 현장에서 촬영한 영상을 VLM 분석 시스템으로 전달한다.
2) VLM은 영상에서 사람의 접근 여부와 인상착의 및 주변 위험 상황을 분석한다.
3) 분석된 정보와 상황을 상황실 작업자에게 제공한다.
4) 사람이 위험지역에 접근하면 해당 지역에서 벗어나도록 안내 문장을 생성한다.
5) 생성된 문장을 AI 음성으로 변환하여 현장 스피커로 전달한다.
6) 스피커를 통해 위험지역 접근자에게 이탈 안내를 출력한다.

| 서비스 흐름도

텔레오퍼레이션 및 모의 위험물 처리



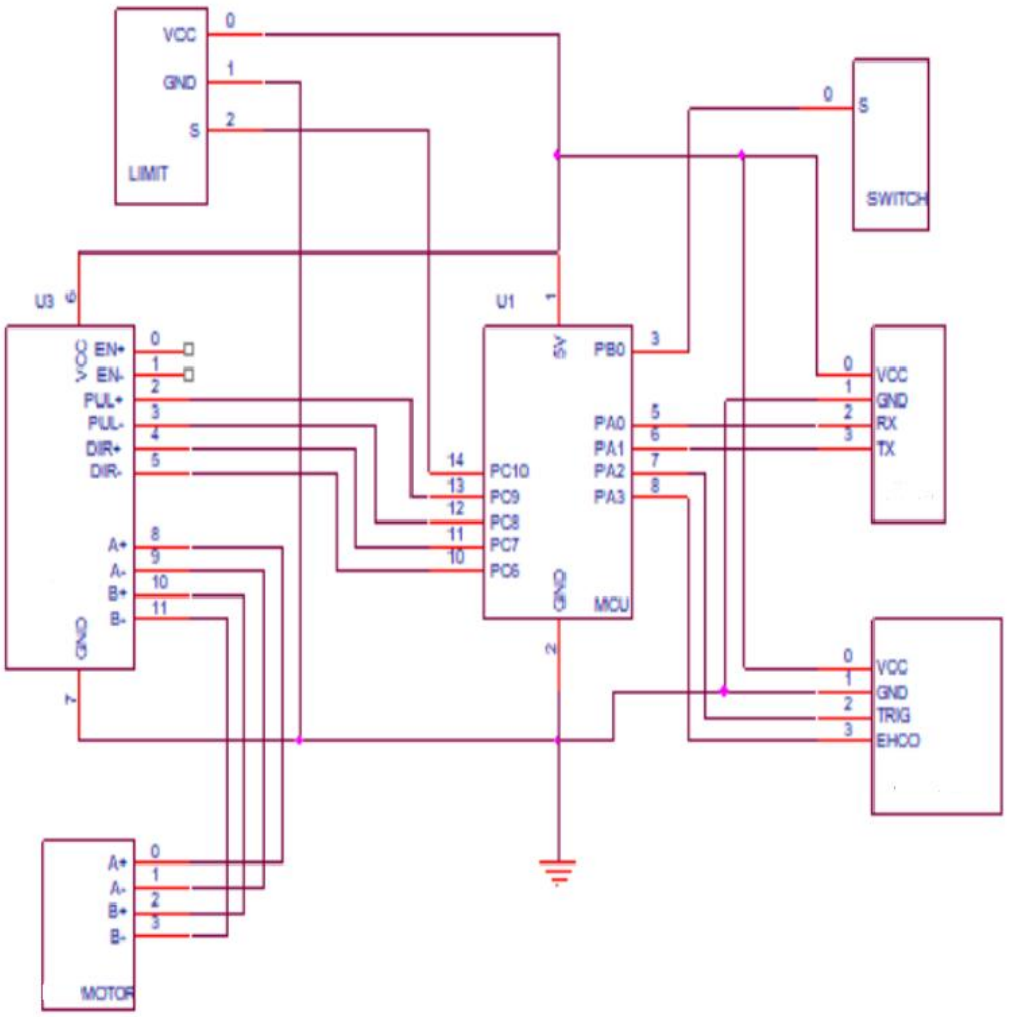
로봇팔 텔레오퍼레이션

1) 작업자는 상황실에서 실시간 현장 영상과 모의 위험물 배선 도면을 확인한다.
2) 마스터 로봇팔을 직접 움직여 원격조작 명령을 입력한다.
3) 상황실 제어 PC는 U2D2를 통해 마스터 로봇팔의 관절 위치값을 취득한다.
4) 취득한 관절값을 네트워크를 통해 현장 제어 PC로 전송한다.
5) 현장 제어 PC는 수신한 관절값을 슬레이브 로봇팔의 제어값으로 변환한다.
6) 슬레이브 로봇팔은 마스터 로봇팔의 움직임을 실시간으로 추종한다.

모의 위험물 원격 처리

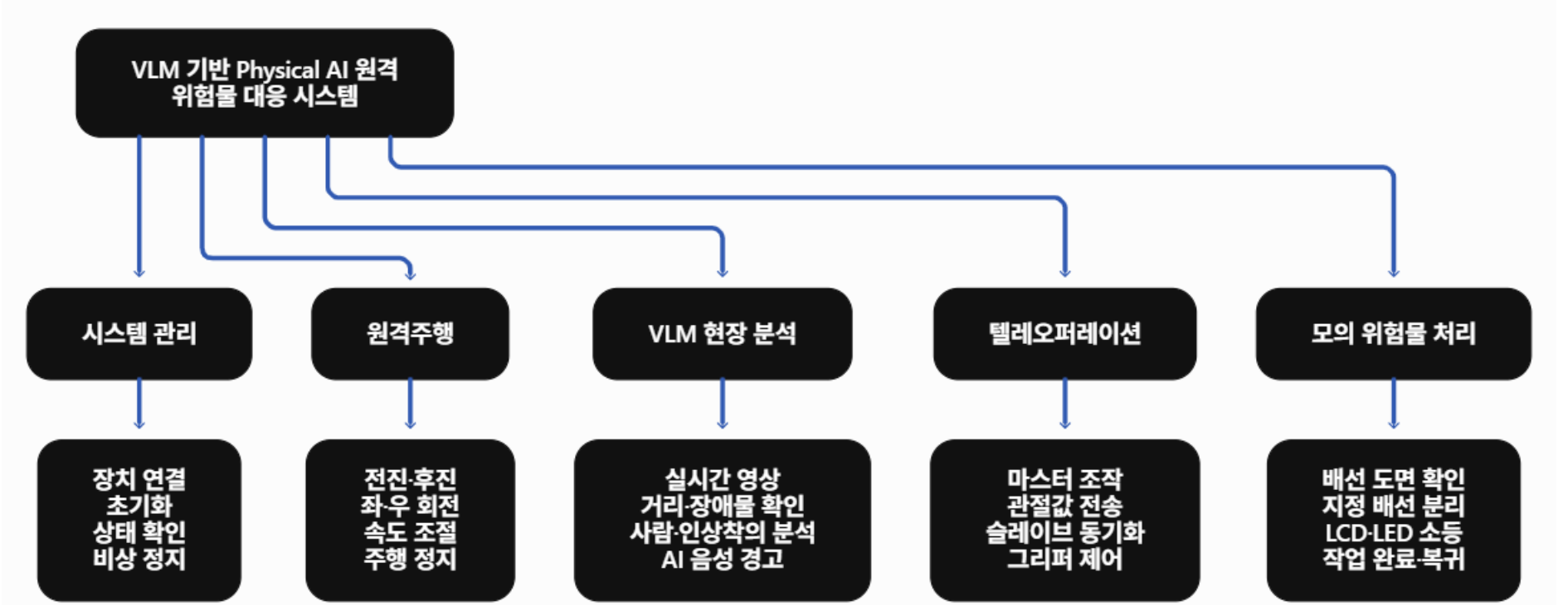
1) 작업자는 현장 영상으로 로봇팔과 제거 대상의 위치를 확인한다.
2) 배선 도면을 참고하여 모의 위험물에서 제거해야 할 배선을 확인한다.
3) 슬레이브 로봇팔을 조작하여 대상에 접근하고 그리퍼로 지정된 배선을 분리한다.
4) 지정된 배선이 분리되면 모의 위험물의 LCD와 LED가 꺼진다.
5) 작업 완료 후 로봇팔과 모바일 플랫폼을 안전 위치로 복귀시킨다.

| 하드웨어/센서 구성도



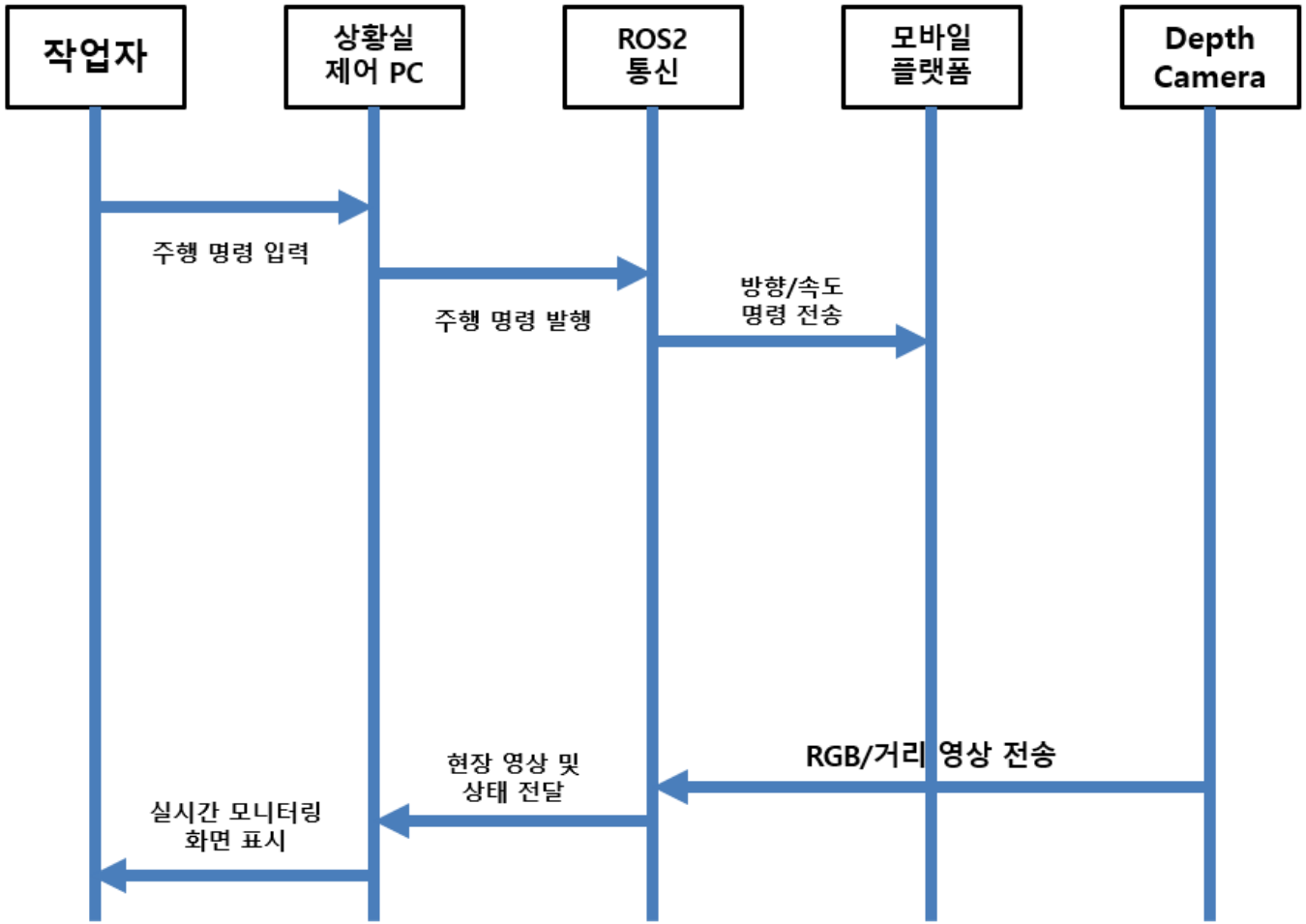
센서 종류	연결 핀	설명
MDD10A	VCC	배터리 양극과 연결하여 모터 구동전원 공급
	GND	배터리 음극 및 제어보드 GND와 공통 연결
	PUL1	1번 DC 모터의 회전속도 제어
	PUL2	2번 DC 모터의 회전속도 제어
	DIR1	1번 DC 모터의 회전방향 제어
	DIR2	2번 DC 모터의 회전방향 제어
DC motor	M1A	1번 DC 모터의 A단자 연결
	M1B	1번 DC 모터의 B단자 연결
	M2A	2번 DC 모터의 A단자 연결
	M2B	2번 DC 모터의 B단자 연결
U2D2	USB	제어 PC와 연결하여 USB 신호를 Dynamixel 통신신호로 변환
	TTL/RS-485	Dynamixel 모터의 통신 및 제어 데이터 송수신
	Power Hub	외부 12.4V 전원을 Dynamixel 모터에 공급

| 메뉴 구성도


VLM 기반 Physical AI 원격
위험물 대응 시스템
시스템 관리
원격주행
VLM 현장 분석
텔레오퍼레이션
모의 위험물 처리
장치 연결
초기화
상태 확인
비상 정지
전진·후진
좌·우 회전
속도 조절
주행 정지
실시간 영상
거리·장애물 확인
사람·인상착의 분석
AI 음성 경고
마스터 조작
관절값 전송
슬레이브 동기화
그리퍼 제어
배선 도면 확인
지정 배선 분리
LCD·LED 소등
작업 완료·복귀

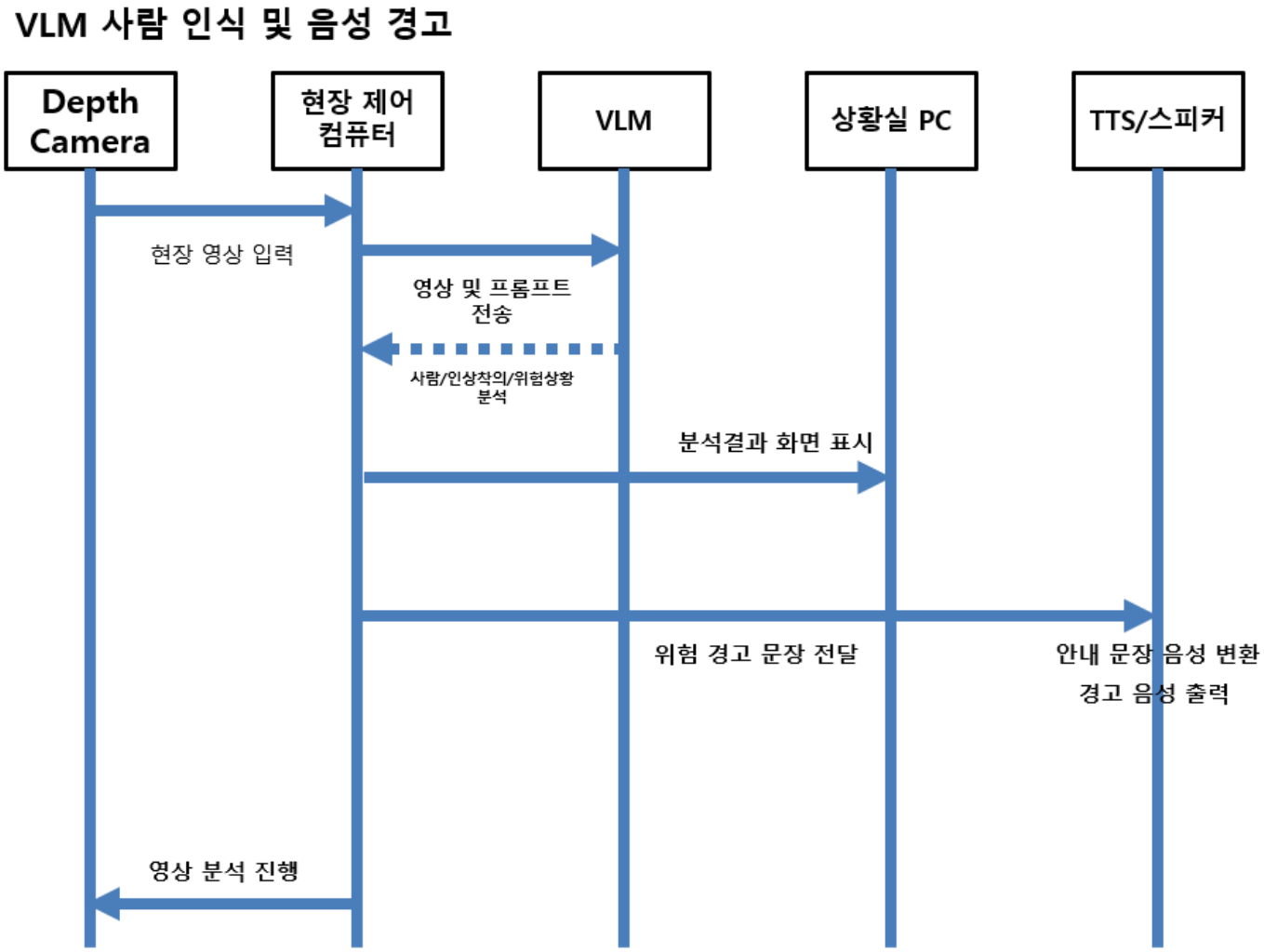
| 기능 처리도(기능 흐름도)

모바일 플랫폼 원격 주행



작업자가 상황실에서 주행 명령을 보내면 모바일 플랫폼에 방향 및 속도 명령이 전달된다.
Depth Camera로 촬영한 현장 영상과 거리 정보는 상황실 화면에 실시간으로 표시된다

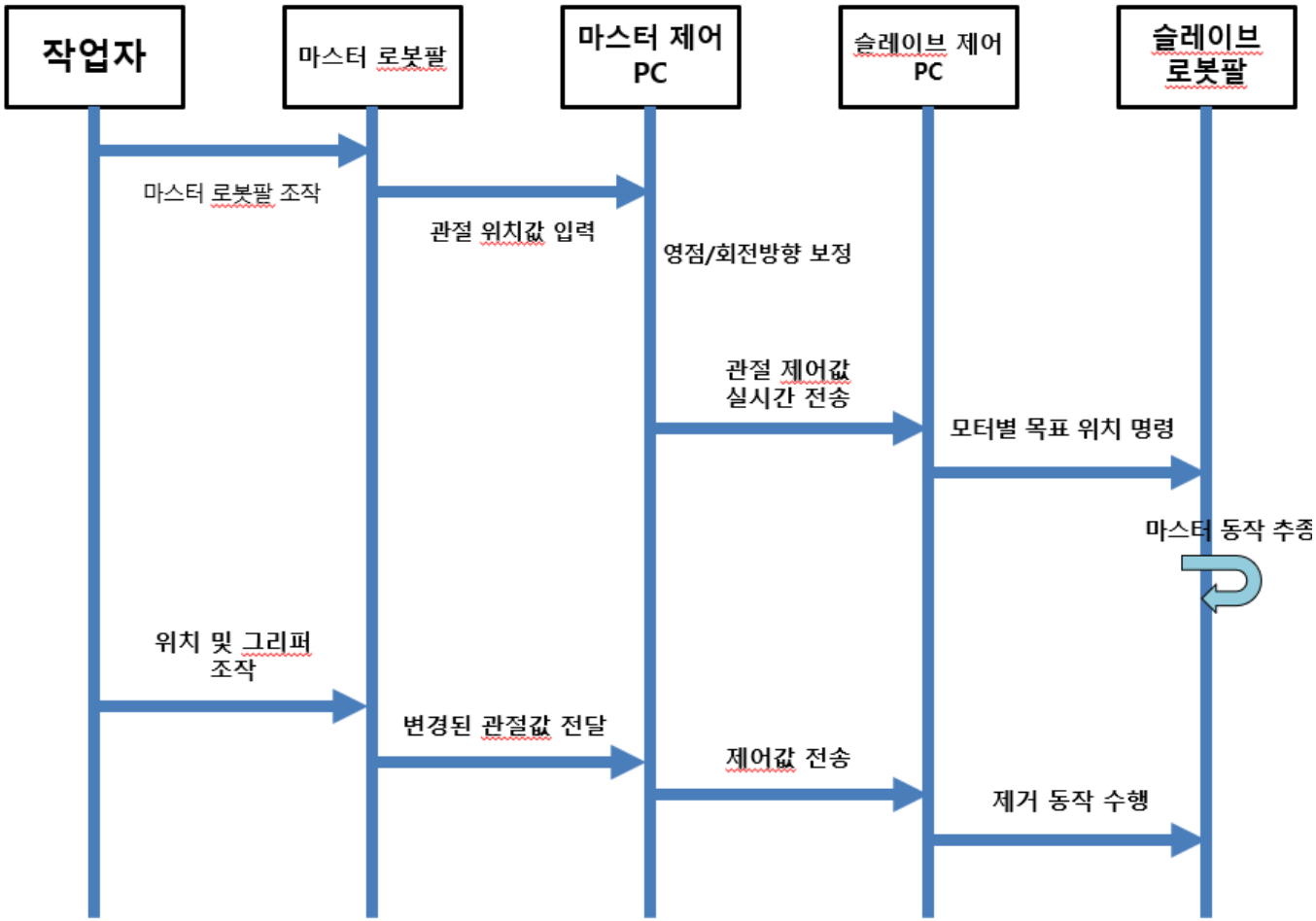
| 기능 처리도(기능 흐름도)



현장 영상은 VLM을 통해 사람의 존재와 인상착의 및 위험상황을 분석한다. 사람이 인식되면 분석 결과를 상황실에 전달하고 생성된 경고 문장을 TTS 음성으로 변환하여 현장 스피커로 출력한다.

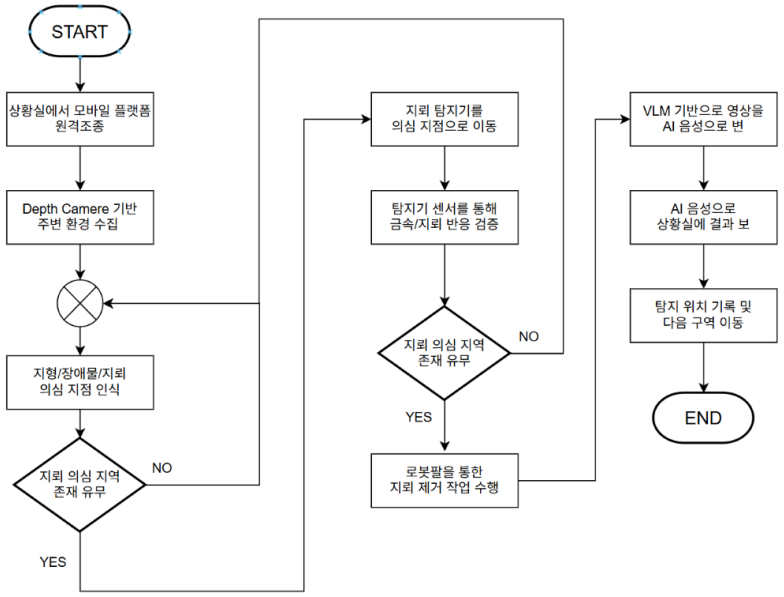
| 기능 처리도(기능 흐름도)

듀얼 로봇팔 텔레오퍼레이션



작업자가 마스터 로봇팔을 움직이면 각 관절 위치값이 보정된 후 슬레이브 제어 PC로 전송된다.
슬레이브 로봇팔은 마스터 움직임을 실시간 추종하여 모의 위험물을 제거하고 원격 제거 작업을 수행한다.

| 알고리즘 명세서



알고리즘 시나리오

1. 시스템을 초기화하고 카메라, 센서, 모바일 플랫폼 및 로봇팔의 연결 상태를 확인한다.
2. 작업자는 상황실에서 모바일 플랫폼을 원격 조작하고 Depth Camera 영상으로 주변 지형과 장애물을 확인한다.
3. 지뢰 의심 지점이 확인되면 해당 위치로 이동하여 위험 매설물/지뢰를 확인한다. 없을 시 다른 구역을 탐색한다.
4. 센서 반응이 확인되면 마스터 로봇팔의 관절값을 슬레이브 로봇팔에 전달하여 모의 지뢰 제거 작업을 수행한다.
5. 작업 중 VLM은 현장 영상에서 사람과 위험 상황을 지속적으로 분석하고 민간인 감지 시 인상착의 안내와 위험 지역 이탈 음성을 출력한다.
6. 제거 결과를 확인한 후 다음 구역으로 이동하거나 모바일 플랫폼을 안전지역으로 복귀시켜 작업을 종료한다.

※ 통신 두절 또는 장치 이상 발생 시 모바일 플랫폼과 로봇팔의 동작을 즉시 정지한다.

| 알고리즘 상세 설명서

1) VLM 기반 사람 인식 및 음성 경고 알고리즘

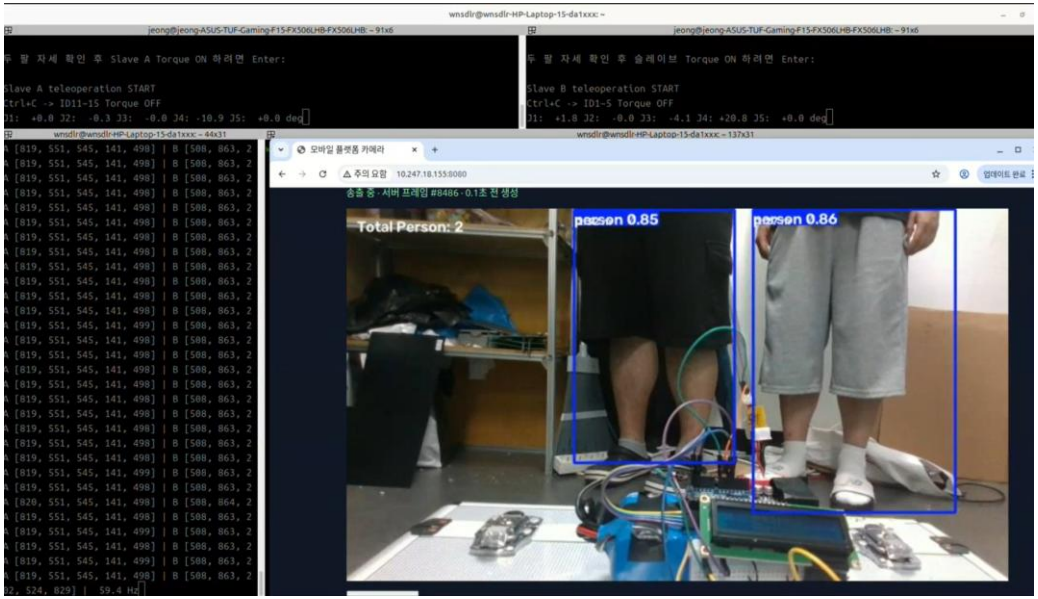
현장 카메라로 취득한 영상 프레임을 VLM API에 전달하여 사람의 존재 여부와 인상착의 및 주변 위험상황을 분석한다. 사람이 인식되면 분석 결과를 바탕으로 위험지역 이탈을 안내하는 경고 문장을 생성한다. 생성된 문장은 TTS 기술을 통해 음성 데이터로 변환되며 현장 스피커로 출력되어 주변 사람에게 위험상황을 안내한다.

영상 프레임 획득 -> VLM API 전송 -> 사람/위험 상황 분석 -> 경고 문장 생성 -> TTS 음성 출력

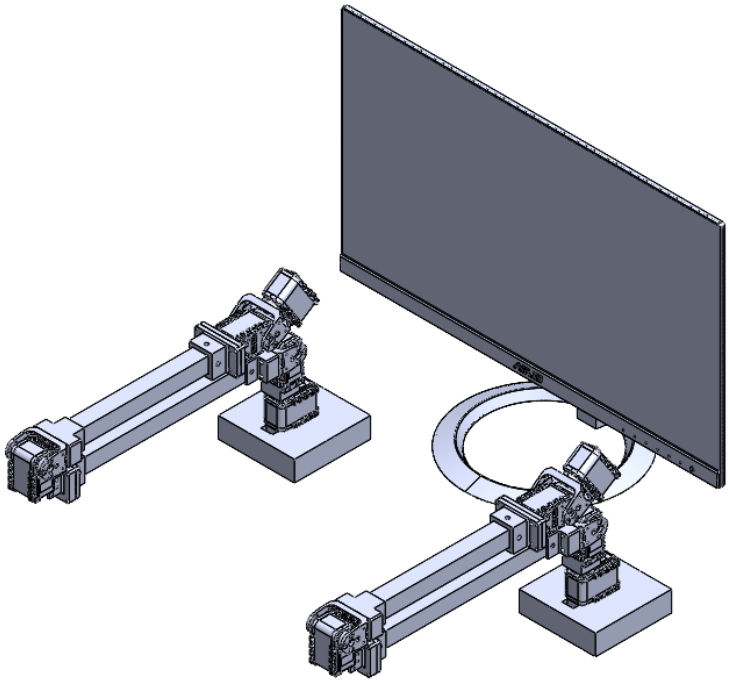
2) 듀얼 로봇팔 텔레오퍼레이션 알고리즘

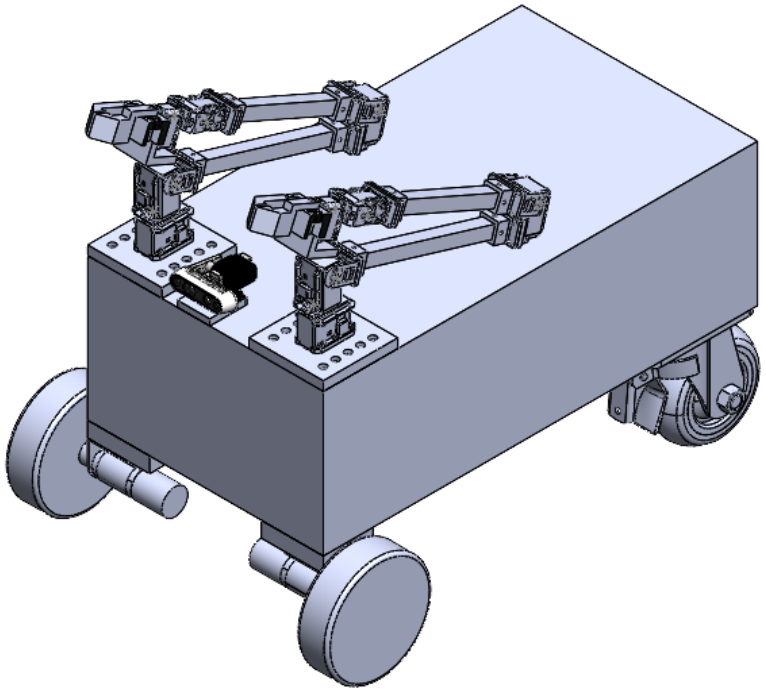
작업자가 마스터 로봇팔을 움직이면 각 관절의 위치값을 실시간으로 취득한다. 취득한 값에 영점과 회전 방향 등의 보정값을 적용한 후 동일한 IP를 통해 슬레이브 제어 PC로 전송한다. 슬레이브 로봇팔은 수신한 관절값을 모터별 목표 위치로 적용하여 마스터 로봇팔의 움직임을 추종하며 작업자는 그리퍼를 조작하여 모의 위험물의 파지 및 제거 작업을 수행한다.

마스터 관절값 취득 -> 영점/방향 보정 -> 제어값 전송 -> 슬레이브 목표값 적용 -> 파지 및 제거



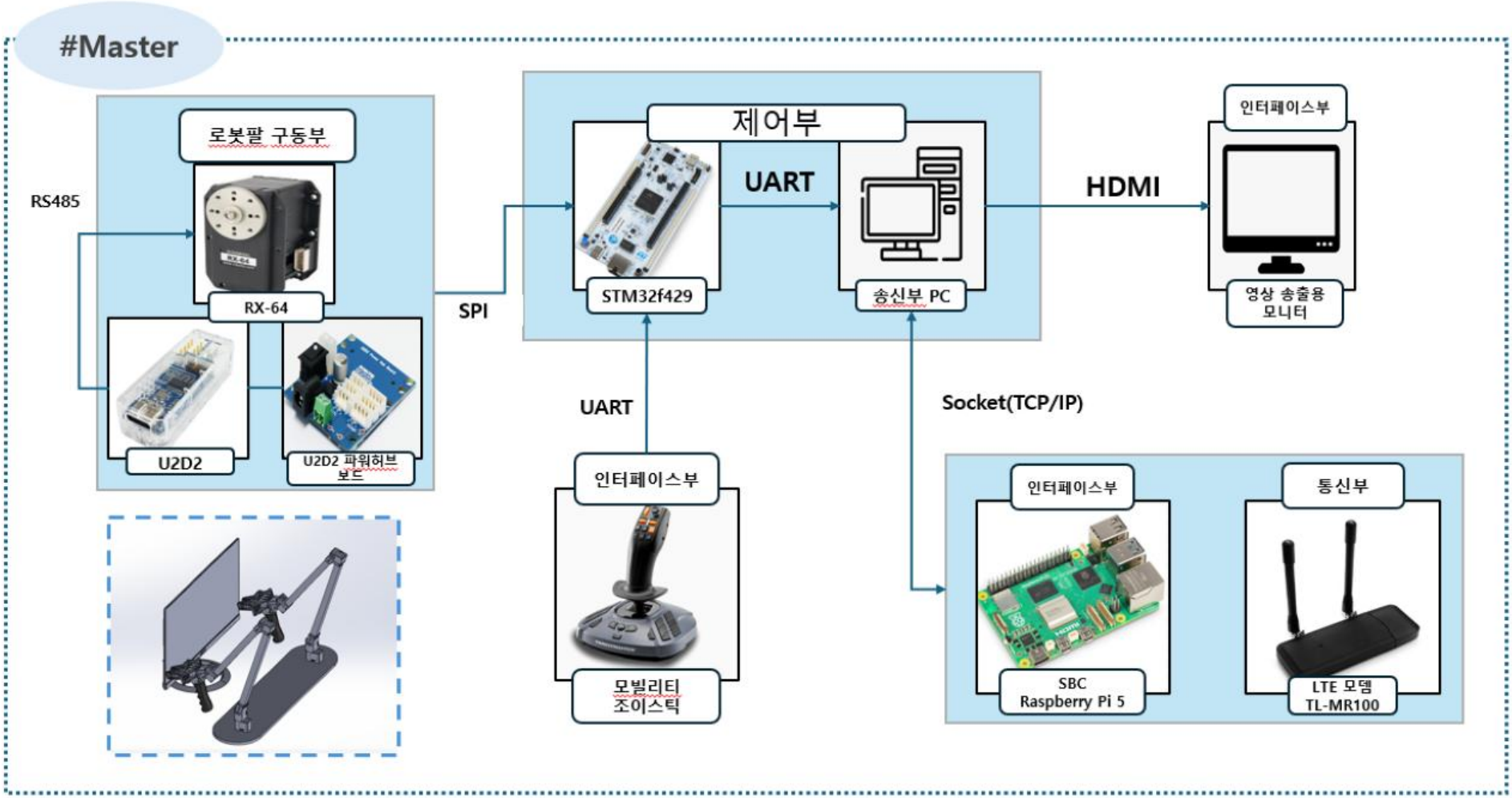
| 하드웨어 설계도



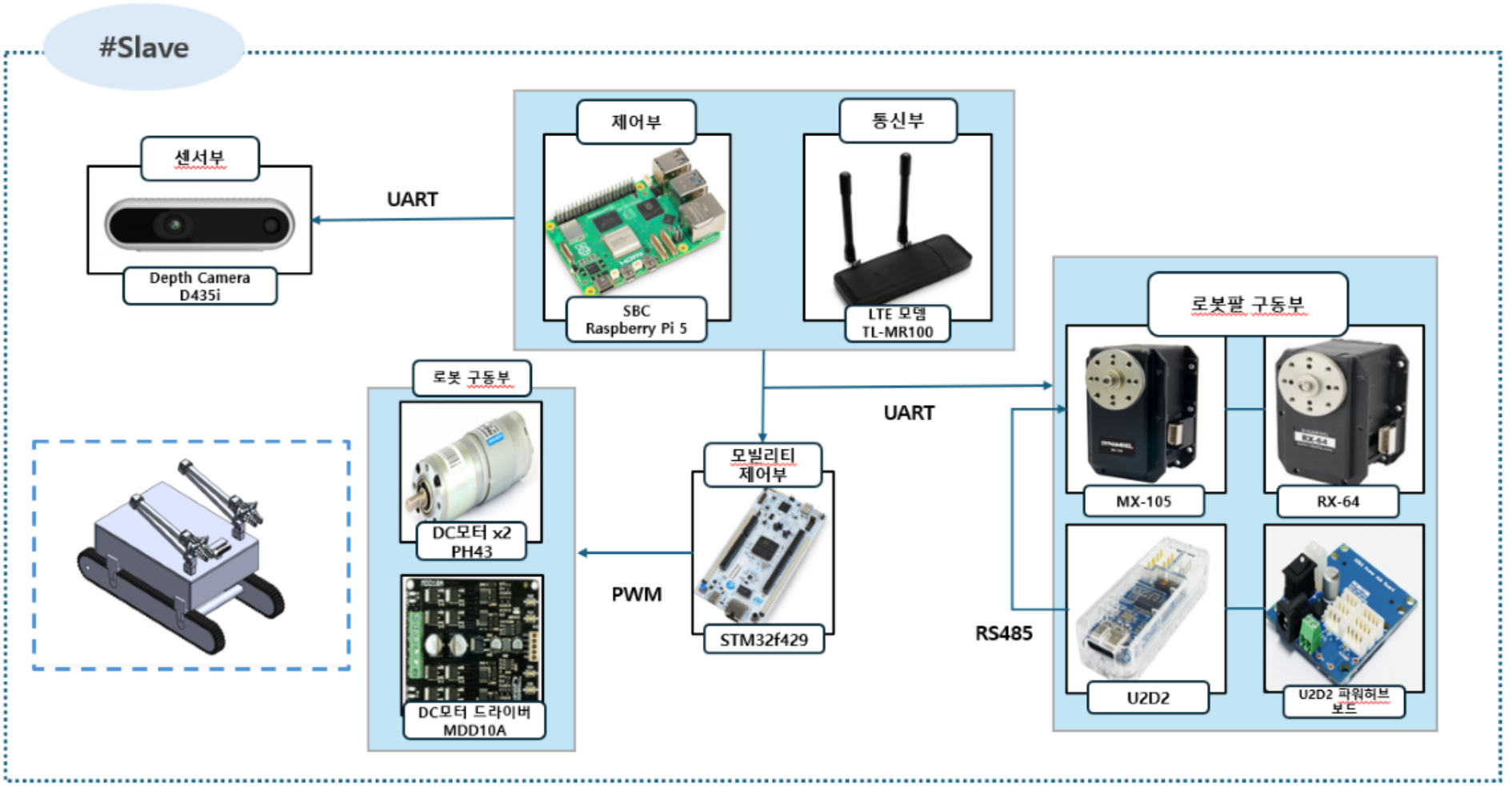


듀얼 마스터-슬레이브 로봇팔 기반 텔레오퍼레이션 하드웨어 설계

| 하드웨어 설계도


#Master
로봇팔 구동부
RS485
RX-64
U2D2
U2D2 파워허브 보드
SPI
제어부
STM32f429
UART
송신부 PC
HDMI
인터페이스부
영상 송출용 모니터
UART
인터페이스부
모빌리티 조이스틱
Socket(TCP/IP)
인터페이스부
SBC Raspberry Pi 5
통신부
LTE 모뎀 TL-MR100

| 하드웨어 설계도


#Slave
제어부
통신부
센서부
UART
Depth Camera
D435i
SBC
Raspberry Pi 5
LTE 모뎀
TL-MR100
로봇팔 구동부
로봇 구동부
UART
모빌리티
제어부
MX-105
RX-64
DC모터 x2
PH43
PWM
STM32f429
RS485
U2D2
U2D2 파워허브
보드
DC모터 드라이버
MDD10A

| 프로그램 - 목록

기능 분류	기능번호	기능 명
SYS	SYS-01-01	전체 시스템 초기화
	SYS-01-02	장치 연결상태 확인
	SYS-01-03	시스템 동작상태 표시
DRV	DRV-02-01	주행 명령 입력
	DRV-02-02	원격 주행 명령 전송
	DRV-02-03	전진·후진 및 좌·우 회전 제어
	DRV-02-04	주행 속도 조절 및 정지
VIS	VIS-03-01	Depth Camera 영상 취득
	VIS-03-02	실시간 현장 영상 전송
	VIS-03-03	거리 및 장애물 정보 확인
VLM	VLM-04-01	현장 영상 VLM 분석
	VLM-04-02	사람 및 인상착의 인식
	VLM-04-03	위험지역 경고 문장 생성
	VLM-04-04	TTS 기반 AI 음성 경고

| 프로그램 - 목록

기능 분류	기능번호	기능 명
TEL	TEL-05-01	마스터 로봇팔 관절값 취득
	TEL-05-02	관절 영점 및 회전 방향 보정
	TEL-05-03	관절 제어값 원격 전송
	TEL-05-04	슬레이브 로봇팔 추종 제어
	TEL-05-05	듀얼 로봇팔 개별 제어
EOD	EOD-06-01	제거 대상 위치 확인
	EOD-06-02	로봇팔 그리퍼 제어
	EOD-06-03	모의 지뢰 파지 및 지정 배선 분리
	EOD-06-04	LCD·LED 작동 해제 확인
COM	COM-07-01	상황실·현장 시스템 통신 연결
	COM-07-02	제어 명령 및 상태정보 송수신
SAF	SAF-08-01	통신 두절 감지 및 동작 정지
	SAF-08-02	로봇팔 관절 동작범위 제한
	SAF-08-03	전체 시스템 비상 정지

| 핵심 소스코드(1)

```python
import asyncio
import edge_tts
import pygame
import io

async def speak_edge(text, voice="ko-KR-SunHiNeural", rate="+0%", pitch="+0Hz"):
    # 1. Edge TTS 통신 객체 생성 (텍스트, 목소리, 속도, 음정 설정)
    communicate = edge_tts.Communicate(text, voice, rate=rate, pitch=pitch)

    # 2. 음성 데이터를 메모리에 모으기
    audio_data = bytearray()
    async for chunk in communicate.stream():
        if chunk["type"] == "audio":
            audio_data.extend(chunk["data"])

    # 3. 하드디스크 저장 없이 메모리 공간(BytesIO)에 담기
    fp = io.BytesIO(audio_data)

    # 4. pygame으로 바로 재생
    pygame.mixer.init()
    pygame.mixer.music.load(fp)
    pygame.mixer.music.play()

    while pygame.mixer.music.get_busy():
        pygame.time.Clock().tick(10)

# 실행 예시
if __name__ == "__main__":
    # 속도 20% 빠르게, 음정 10Hz 높게 설정
    asyncio.run(speak_edge("검은색 옷을 입으신 분들 지뢰 위험 구역입니다, 빠르게 주변을 벗어나 주세요", rate="+20%", pitch="+10Hz"))
```

VLM 분석 결과를 AI 음성으로 변환/출력하는 TTS 핵심코드

| 핵심 소스코드(2)

```
# 설정
# ========================
WATCH_DIR = "./content/inbox"
PROCESSED_DIR = "./content/processed"

GEMINI_MODEL = "gemini-3.5-flash"


# ========================
# Edge TTS 설정
# ========================
TTS_VOICE = "ko-KR-SunHiNeural"

# 음성 속도
TTS_RATE = "+20%"

# 음정
TTS_PITCH = "+10Hz"


# ========================
# Gemini API
# ========================
client = genai.Client(
    api_key="AQ.Ab8RN6LrbzGpP0TwbM_AUmtsaQGRNtq2I_Cxin3RHv2-FbvVgw"
)
```

VLM 분석 결과를 AI 음성으로 변환/출력하는 TTS 핵심코드

| 핵심 소스코드(3)

```
        drive_buttons = buttons & DRIVE_BUTTON_MASK;

        switch (drive_buttons)
        {
            case BUTTON_FORWARD:
                left_speed = DRIVE_TEST_SPEED;
                right_speed = DRIVE_TEST_SPEED;
                break;

            case BUTTON_BACKWARD:
                left_speed = -DRIVE_TEST_SPEED;
                right_speed = -DRIVE_TEST_SPEED;
                break;

            case BUTTON_LEFT:
                left_speed = -DRIVE_TEST_SPEED;
                right_speed = DRIVE_TEST_SPEED;
                break;

            case BUTTON_RIGHT:
                left_speed = DRIVE_TEST_SPEED;
                right_speed = -DRIVE_TEST_SPEED;
                break;

            default:
                /*
                 * 버튼을 누르지 않았거나 이동 버튼을
                 * 동시에 여러 개 누른 경우 정지한다.
                 */
                left_speed = 0;
                right_speed = 0;
                break;
        }

        DriveController_Apply(
            left_speed,
            right_speed
        );
```

```
self.sender = MobileSerialSender(port)

self.latest_buttons = 0
self.last_message_time = None
self.command_timed_out = True
self.last_sent_buttons = None

self.subscription = self.create_subscription(
    UInt8,
    TOPIC_NAME,
    self.receive_buttons,
    1,
)

self.timer = self.create_timer(
    UART_SEND_PERIOD_SECONDS,
    self.send_uart_command,
)

self.get_logger().info(
    f"Mobile UART opened: {port}"
)

self.get_logger().info(
    f"Subscribing to: {TOPIC_NAME}"
)
```

모바일 플랫폼 원격주행 제어 및 통신 핵심 코드

| 핵심 소스코드(4)

```python
class DualMasterPublisher(Node):
    def __init__(self) -> None:
        super().__init__("dual_master_publisher")
        self.arm_a_joint_pub = self.create_publisher(
            JointState, "/teleop/master_a/joint_states", 10
        )
        self.arm_a_raw_pub = self.create_publisher(
            Int32MultiArray, "/teleop/master_a/raw_positions", 10
        )
        self.arm_b_joint_pub = self.create_publisher(
            JointState, "/teleop/master_b/joint_states", 10
        )
        self.arm_b_raw_pub = self.create_publisher(
            Int32MultiArray, "/teleop/master_b/raw_positions", 10
        )

    def publish_arm(self, arm_name: str, raw_values: list[int], models: list[int]) -> None:
        joint_msg = JointState()
        joint_msg.header.stamp = self.get_clock().now().to_msg()
        joint_msg.name = [f"joint{i}" for i in range(1, 6)]

        positions_rad = []
        for raw, model in zip(raw_values, models):
            _, raw_max, range_deg = MODEL_INFO[model]
            angle_deg = raw * range_deg / raw_max
            positions_rad.append(math.radians(angle_deg))
        joint_msg.position = positions_rad

        raw_msg = Int32MultiArray()
        raw_msg.data = raw_values

        if arm_name == "A":
            self.arm_a_joint_pub.publish(joint_msg)
            self.arm_a_raw_pub.publish(raw_msg)
        else:
            self.arm_b_joint_pub.publish(joint_msg)
            self.arm_b_raw_pub.publish(raw_msg)
```

마스터 로봇팔의 관절 정보를 ROS2 토픽으로 변환하여 슬레이브 시스템에 전달하는 핵심 통신 코드

| 핵심 소스코드(5)

```python
def read_position(port, packet, dxl_id, retries=2):
    last_error = "unknown"
    for _ in range(retries + 1):
        raw, comm_result, dxl_error = packet.read2ByteTxRx(
            port, dxl_id, ADDR_PRESENT_POSITION
        )
        if comm_result == COMM_SUCCESS and dxl_error == 0:
            return int(raw)
        if comm_result != COMM_SUCCESS:
            last_error = packet.getTxRxResult(comm_result)
        else:
            last_error = packet.getRxPacketError(dxl_error)
        time.sleep(0.001)
    raise RuntimeError(f"ID {dxl_id} 위치 읽기 실패: {last_error}")

def read_arm(arm, packet):
    return [read_position(arm.port, packet, dxl_id) for dxl_id in arm.ids]
```

DYNAMIXEL 엔코더의 현재 위치를 실시간으로 읽어 마스터 로봇팔의 움직임을 감지하는 코드

| 핵심 소스코드(6)

```python
    def publish_arm(self, arm_name: str, raw_values: list[int], models: list[int]) -> None:
        joint_msg = JointState()
        joint_msg.header.stamp = self.get_clock().now().to_msg()
        joint_msg.name = [f"joint{i}" for i in range(1, 6)]

        positions_rad = []
        for raw, model in zip(raw_values, models):
            _, raw_max, range_deg = MODEL_INFO[model]
            angle_deg = raw * range_deg / raw_max
            positions_rad.append(math.radians(angle_deg))
        joint_msg.position = positions_rad

        raw_msg = Int32MultiArray()
        raw_msg.data = raw_values

        if arm_name == "A":
            self.arm_a_joint_pub.publish(joint_msg)
            self.arm_a_raw_pub.publish(raw_msg)
        else:
            self.arm_b_joint_pub.publish(joint_msg)
            self.arm_b_raw_pub.publish(raw_msg)
```

Raw encoder 값을 실제 관절각으로 변환하여 ROS2에서 활용 가능한 데이터로 가공하는 코드

| 핵심 소스코드(7)

```
271         while not stop_requested and rclpy.ok():
272             now = time.monotonic()
273
274             future_a = executor.submit(read_arm, arm_a, packet)
275             future_b = executor.submit(read_arm, arm_b, packet)
276
277             try:
278                 current_a = future_a.result()
279                 last_a = current_a
280             except Exception as exc:
281                 node.get_logger().warning(f"Arm A read error: {exc}")
282                 current_a = None
283
284             try:
285                 current_b = future_b.result()
286                 last_b = current_b
287             except Exception as exc:
288                 node.get_logger().warning(f"Arm B read error: {exc}")
289                 current_b = None
290
291             if current_a is not None:
292                 node.publish_arm("A", current_a, arm_a.models)
293             if current_b is not None:
294                 node.publish_arm("B", current_b, arm_b.models)
295
296             rclpy.spin_once(node, timeout_sec=0.0)
297
298             if now >= next_torque_guard:
299                 for arm in (arm_a, arm_b):
300                     for dxl_id in arm.ids:
301                         try:
302                             torque_off(arm.port, packet, dxl_id)
303                         except Exception:
304                             pass
305                 next_torque_guard = now + 1.0
```

두 개의 마스터 팔을 병렬로 읽고 일정 주기로 송신하여 텔레오퍼레이션을 수행하는 핵심 실행 코드

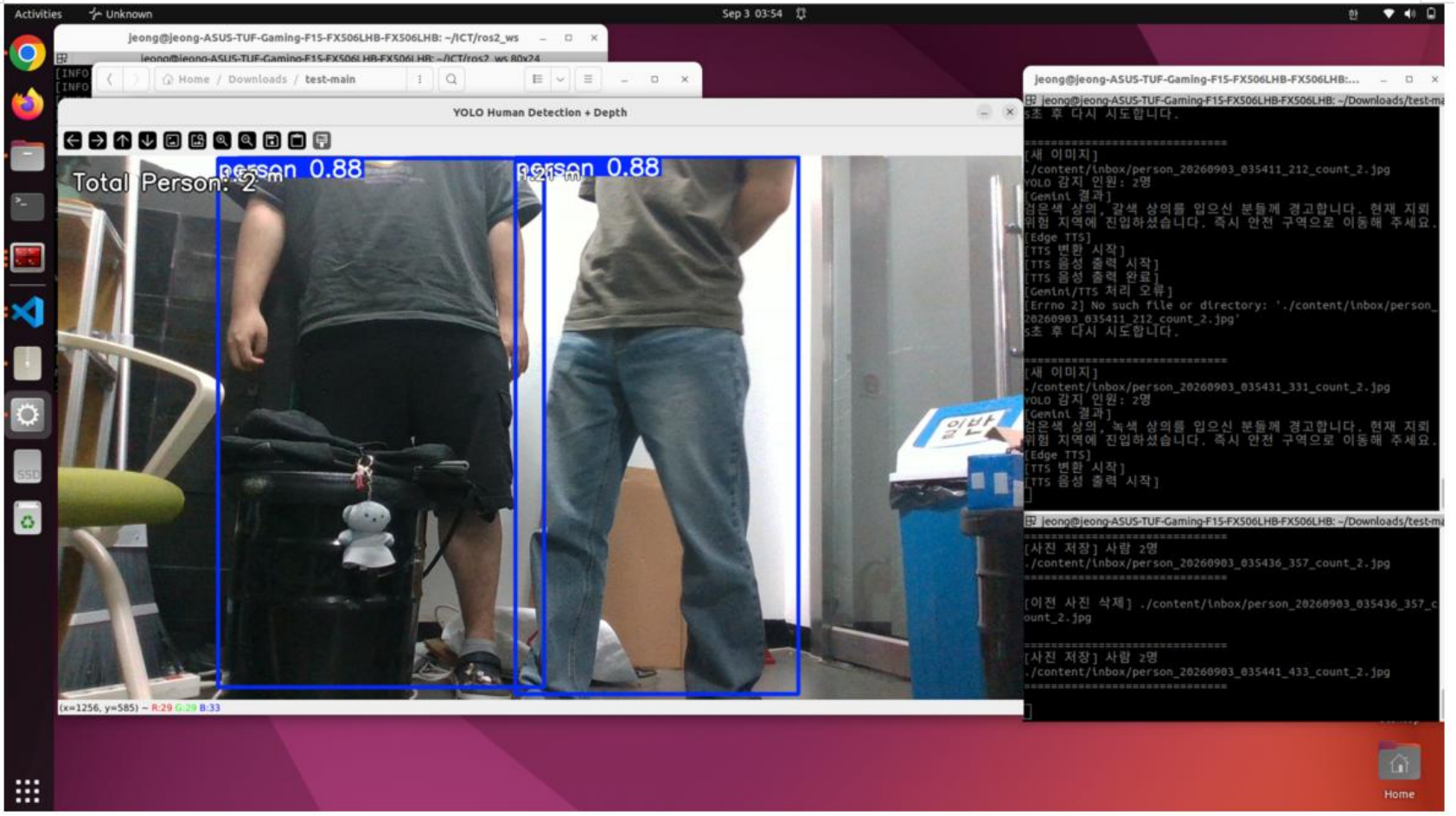
| 참조-개발 환경 및 설명

구분	항목	적용내역
S/W 개발환경	Ubuntu Linux	상황실과 현장 제어 시스템의 로봇 제어 프로그램 실행 운영체제
	ROS 2	주행/영상/센서/로봇팔 제어 데이터를 노드 단위로 송수신하기 위한 로봇 미들웨어
	Visual Studio Code	Python 및 C/C++ 기반 제어 프로그램 작성과 디버깅에 사용
	Python 3	VLM 분석, 원격 통신, 영상처리 및 로봇팔 텔레오퍼레이션 프로그램 개발
	C/C++	모바일 플랫폼과 마이크로컨트롤러의 모터 및 장치 제어에 사용
	OpenCV	카메라 영상 취득, 전처리 및 상황실 영상 출력에 사용
	Intel RealSense SDK	D435i의 RGB 영상과 Depth 거리정보를 취득하기 위해 사용
	VLM API	현장 영상에서 사람의 인상착의와 위험 상황을 분석하고 안내 문장을 생성
	TTS 엔진	VLM이 생성한 위험 안내 문장을 AI 음성으로 변환하여 출력
	Dynamixel SDK	U2D2를 통해 마스터·슬레이브 로봇팔의 관절값과 제어 명령을 송수신
	STM32CubeIDE	모바일 플랫폼의 모터 및 주변장치 제어 프로그램 개발
	Git·GitHub	프로그램 버전관리, 코드 공유 및 팀원별 개발 결과 통합

| 참조-개발 환경 및 설명

구분	항목	적용내역
H/W 개발환경	상황실 제어 PC·모니터	현장 영상과 VLM 분석 결과를 확인하고 원격 제어 프로그램을 실행
	마스터 로봇팔 2대	작업자의 움직임을 관절 위치값으로 취득하여 현장 로봇팔 조작에 사용
	슬레이브 로봇팔 2대	마스터 로봇팔의 움직임을 추종하여 모의 위험물 처리 작업을 수행
	Dynamixel 모터	마스터/슬레이브 로봇팔의 관절 구동과 위치 제어에 사용
	U2D2·파워허브	PC와 Dynamixel 모터 간 통신 및 로봇팔 전원 공급에 사용
	모바일 플랫폼	작업자를 대신하여 위험지역으로 이동하고 현장 장비를 운반
	DC 모터·모터 드라이버	모바일 플랫폼의 전진/후진/회전/속도를 제어
	Raspberry Pi 5	카메라/센서 정보 처리와 현장 장치의 통합 제어에 사용
	STM32	모바일 플랫폼의 주행모터와 주변장치의 실시간 제어에 사용
	Intel RealSense D435i	현장 RGB 영상과 주변 물체까지의 거리정보를 취득

| 참조-S/W 기능 실사 사진



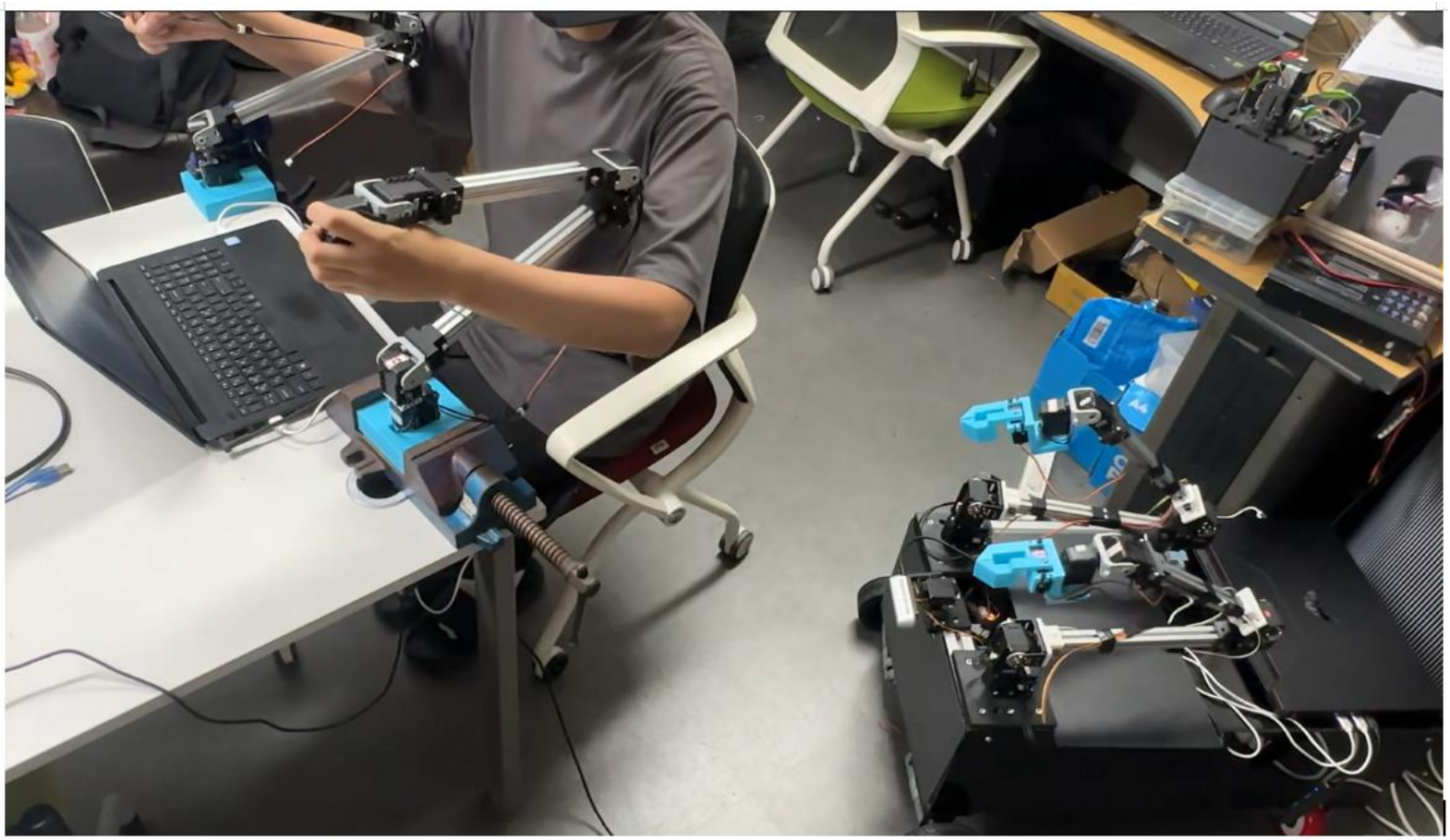
카메라 영상에서 사람과 주변 상황을 분석하고 위험지역에 사람이 접근하면
인상착의를 안내한 뒤 현장에서 벗어나도록 AI 음성으로 경고함

| 참조-S/W 기능 실사 사진



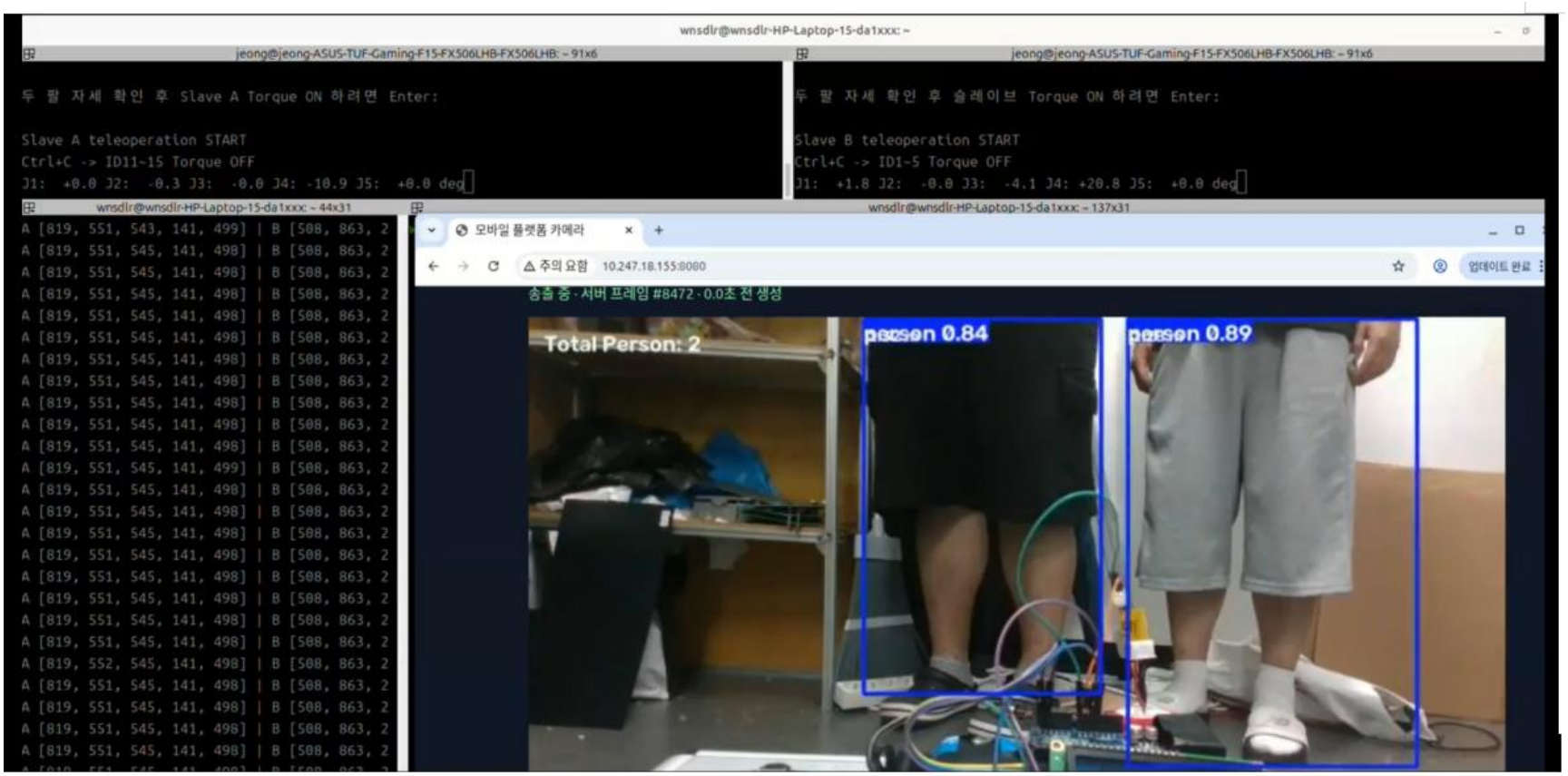
상황실의 조작 명령을 현장 플랫폼에 전달하여 실시간으로 원격 제어함

| 참조-S/W 기능 실사 사진



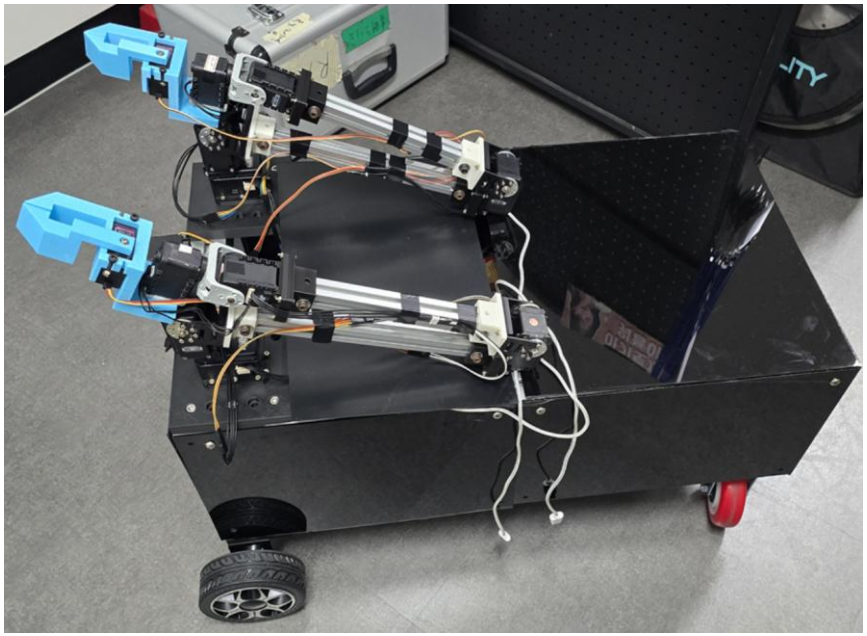
마스터 로봇팔의 관절 움직임을 슬레이브 로봇팔에
실시간으로 전달하여 모의 지뢰 탐지 및 제거 작업을 원격으로 수행함

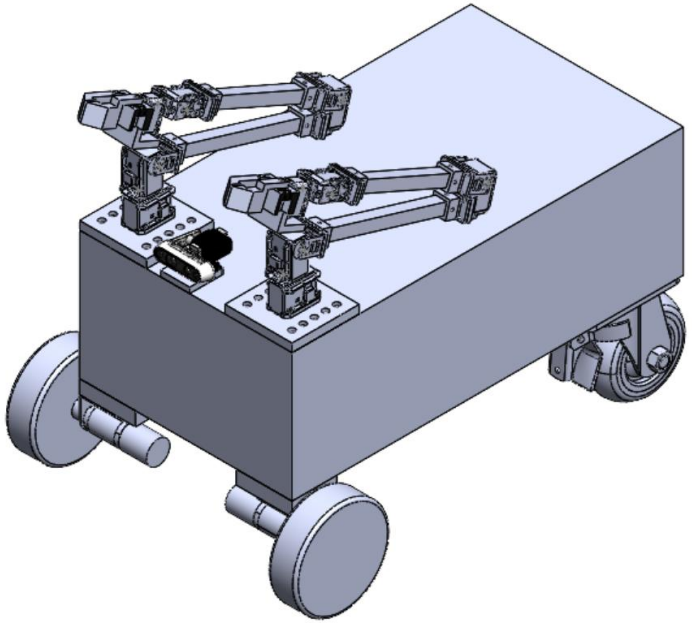
| 참조-S/W 기능 실사 사진



현장 카메라 영상과 센서 정보를 상황실로 전송하여 로봇의 주변 환경과 작업 상태를 실시간으로 확인함

| 참조-H/W 기능 실사 사진

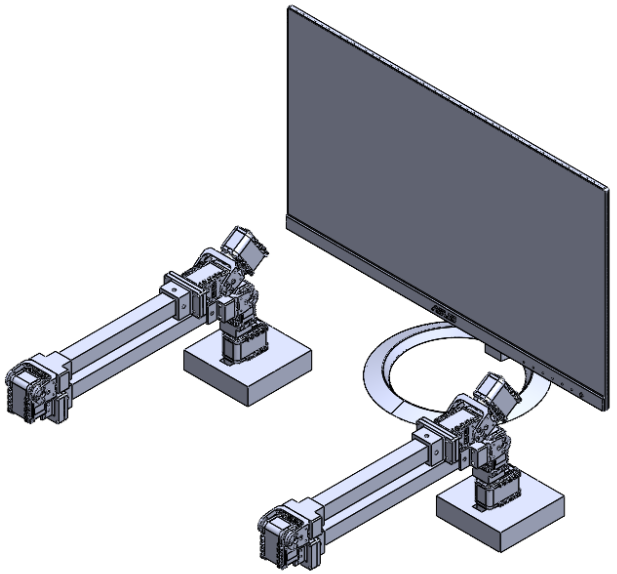




3D 설계 도면을 기반으로 구현한 듀얼 로봇팔 모바일 플랫폼

| 참조-H/W 기능 실사 사진





3D 설계 도면을 기반으로 구현한 듀얼 마스터 로봇팔 원격조작 시스템

| 참조-프로젝트 관리

프로젝트 관리

VLM 기반 Physical AI 지뢰탐지 로봇

- Slack 채널:
- 멘토: 조만재
- 팀장: 이정환
- 팀원: 곽태민 문준익 윤서연 이태훈
- 이메일

이정환: jeonghwan1376@gmail.com

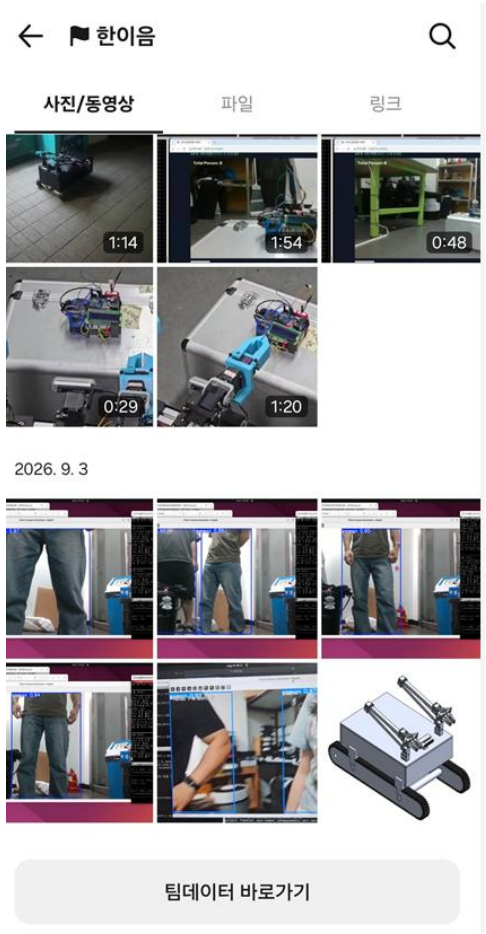
곽태민: frank0526@naver.com

문준익: wnsdlr1121@naver.com

윤서연: yunseoyeon59@gmail.com

이태훈: lthsty79@gmail.com

프로젝트 소개
작업 현황
회의록
코드 공유용 임시 페이지
TTs 코드
VLM 코드



Notion과 카카오톡을 활용한 일정/회의록 관리 및 프로젝트 자료 공유

| 참조-프로젝트 관리

이슈 관리



ZOOM을 통한 정기적인 멘토링 회의

Thank you

